Artlificial Intelligence

Planning

Christopher Simpkins

Kennesaw State University

1 Classical Planning

Classical planning is the task of finding a sequence of actions to accomplish a goal in a discrete, deterministic, static, fully observable environment. Two previous approaches:

- Graph search, e.g., A^*
- Hybrid propositional logical agent

Two limitations:

- Require ad-hoc heuristics
- Require explicit representation of exponentially large state space.

Planning Domain Definition Language solves these problems using a factored representation based on first-order logic.

- A **state** is a conjunction of ground atomic fluents – single predicates containing no variables.
  - $At(Truck_1, Melbourne)$ is a ground atomic fluent, $At(t_1, from)$ is not.
- PDDL uses **database semantics**, or the **closed-world assumption**: any fluents not mentioned are false, and unique names represent distinct objects.

2 Planning Domain Definition Language (PDDL)

Action schema is a family of ground actions.

- Action name and list of variables
- Precondition: conjunction of literals
  - Action a is **applicable** in state s if $s \models a.precondition$
- Effect: conjunction of literals
  - **Result** of executing action a in state s is s' is applying delete list and add list to s:

* $DEL(a)$, delete list: remove negative literals in action's effects.
* $ADD(a)$, add list: add positive literals in action's effects.

Action schema:

$$\begin{aligned}&Action(Fly(p, from, to),\\&\quad PRECOND: At(p, from) \wedge Plane(p) \wedge Airport(from) \wedge Airport(to)\\&\quad EFFECT: \neg At(p, from) \wedge At(p, to))\end{aligned}$$

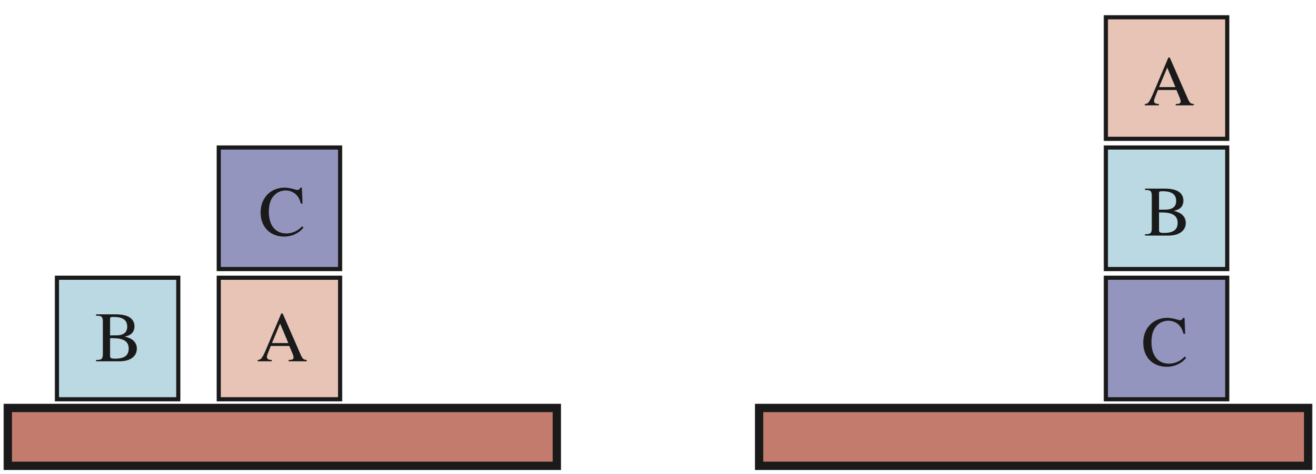
Ground (variable-free) action:

$$\begin{aligned}&Action(Fly(P_1, SFO, JFK),\\&\quad PRECOND: At(P_1, SFO) \wedge Plane(P_1) \wedge Airport(SFO) \wedge Airport(JFK)\\&\quad EFFECT: \neg At(P_1, SFO) \wedge At(P_1, JFK))\end{aligned}$$

3 Air Cargo Transport

$$\begin{aligned}&Init(At(C_1, SFO) \wedge At(C_2, JFK) \wedge At(P_1, SFO) \wedge At(P_2, JFK)\\&\quad \wedge Cargo(C_1) \wedge Cargo(C_2) \wedge Plane(P_1) \wedge Plane(P_2)\\&\quad \wedge Airport(JFK) \wedge Airport(SFO))\\&Goal(At(C_1, JFK) \wedge At(C_2, SFO))\\&Action(Load(c, p, a),\\&\quad \text{PRECOND: } At(c, a) \wedge At(p, a) \wedge Cargo(c) \wedge Plane(p) \wedge Airport(a)\\&\quad \text{EFFECT: } \neg At(c, a) \wedge In(c, p))\\&Action(Unload(c, p, a),\\&\quad \text{PRECOND: } In(c, p) \wedge At(p, a) \wedge Cargo(c) \wedge Plane(p) \wedge Airport(a)\\&\quad \text{EFFECT: } At(c, a) \wedge \neg In(c, p))\\&Action(Fly(p, from, to),\\&\quad \text{PRECOND: } At(p, from) \wedge Plane(p) \wedge Airport(from) \wedge Airport(to)\\&\quad \text{EFFECT: } \neg At(p, from) \wedge At(p, to))\end{aligned}$$

4 Blocks World



Start State

Goal State

5 Blocks World PDDL

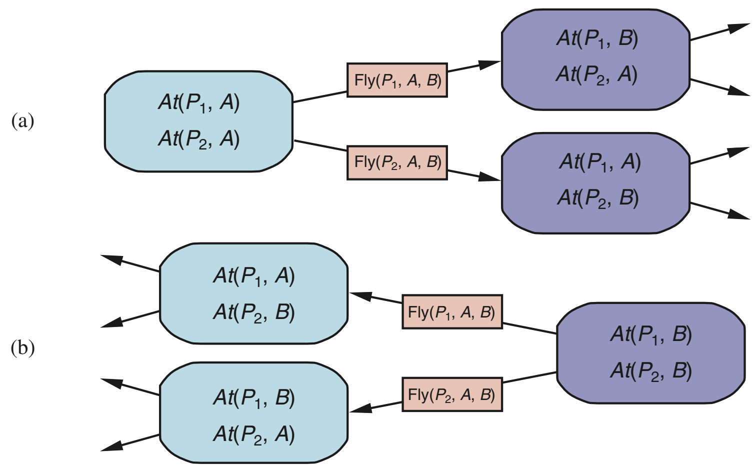
$$
\begin{aligned}
&\mathit{Init}(\mathit{On}(A,\mathit{Table}) \wedge \mathit{On}(B,\mathit{Table}) \wedge \mathit{On}(C,A) \\
&\quad \wedge \mathit{Block}(A) \wedge \mathit{Block}(B) \wedge \mathit{Block}(C) \wedge \mathit{Clear}(B) \wedge \mathit{Clear}(C) \wedge \mathit{Clear}(\mathit{Ta} \\
&\mathit{Goal}(\mathit{On}(A,B) \wedge \mathit{On}(B,C)) \\
&\mathit{Action}(\mathit{Move}(b,x,y), \\
&\quad \textsc{Precond}{:}\ \mathit{On}(b,x) \wedge \mathit{Clear}(b) \wedge \mathit{Clear}(y) \wedge \mathit{Block}(b) \wedge \mathit{Block}(y) \wedge \\
&\qquad\qquad (b \neq x) \wedge (b \neq y) \wedge (x \neq y), \\
&\quad \textsc{Effect}{:}\ \mathit{On}(b,y) \wedge \mathit{Clear}(x) \wedge \neg \mathit{On}(b,x) \wedge \neg \mathit{Clear}(y)) \\
&\mathit{Action}(\mathit{MoveToTable}(b,x), \\
&\quad \textsc{Precond}{:}\ \mathit{On}(b,x) \wedge \mathit{Clear}(b) \wedge \mathit{Block}(b) \wedge \mathit{Block}(x), \\
&\quad \textsc{Effect}{:}\ \mathit{On}(b,\mathit{Table}) \wedge \mathit{Clear}(x) \wedge \neg \mathit{On}(b,x))
\end{aligned}
$$

6 Classical Planning Algorithms

- Forward state space search
- Backward state space search
- SATPlan – boolean satisfiability planning
  - Translate PDDL into propositional form, use a SAT solver
- Graphplan
  - Encode constraints related to preconditions and effects in a **planning graph**.
- Situation calculus
- Constraint satisfaction
- Partial-order planning
  - $Remove(Spare, Trunk)$ and $Remove(Flat, Axle)$ must come before $PutOn(Spare, Axle)$, but removals can happen in any order.

7 Forward and Backward State Space Planning

- Forward search: unify current state against preconditions of each action schema – **applicable** actions.
- Backward search: unify goal states against effects of action schemas – **relevant** actions.



8 Hierarchical Planning

Hierarchical task network plans are built from:

- primitive actions, and
- high-level actions (HLA).

HLAs have one or more **refinements**.

- Refinements may contain other HLAs.
- A refinement with only primitive actions is an **implementation**.
- An HLA achieves a goal if at least one of its implementations achieves the goal.

Here are two goal-achieving implementations for the $Go(Home, SFO)$ HLA:

Refinement(*Go*(*Home*, *SFO*),
  STEPS: [*Drive*(*Home*, *SFOLongTermParking*),
      Shuttle(*SFOLongTermParking*, *SFO*)])
Refinement(*Go*(*Home*, *SFO*),
  STEPS: [*Taxi*(*Home*, *SFO*)])

Refinements can be produced recursivley, as shown in this vacuum world navigation example:

Refinement(*Navigate*([a,b], [x,y]),
  PRECOND: $a = x \wedge b = y$
  STEPS: [])
Refinement(*Navigate*([a,b], [x,y]),
  PRECOND: *Connected*([a,b], [$a-1,b$])
  STEPS: [*Left*, *Navigate*([$a-1,b$], [x,y])])
Refinement(*Navigate*([a,b], [x,y]),
  PRECOND: *Connected*([a,b], [$a+1,b$])
  STEPS: [*Right*, *Navigate*([$a+1,b$], [x,y])])
...

9 Closing Thoughts

- Fun to create toy worlds and solve them.
  - Look up "Monkey and bananas" problem.
- Still have knowledge-acquisition bottleneck.
- Still have problem of specifying large number of rules and facts for non-trivial problems.
- Still have problem of uncertainty – nondeterministic actions and partial observability.

In rest of course, we address these issues with uncertain reasoning and machine learning.